

EX NO : 11

PERCEPTRON

DATE : 21-08-26

AIM:

To implement a binary Perceptron classifier in Python from scratch without external libraries as well as using Scikit-Learn.

CODE:

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.linear_model import Perceptron

# Generate a linearly separable binary dataset
X, y = make_blobs(n_samples=200, centers=2, cluster_std=1.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Scratch Implementation ---
class PerceptronScratch:
    def __init__(self, learning_rate=0.01, epochs=20):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def fit(self, X, y):
        n_features = X.shape[1]
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for _ in range(self.epochs):
            for i, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                prediction = 1 if linear_output >= 0 else 0
                error = y[i] - prediction
                self.weights += self.lr * error * x_i
                self.bias += self.lr * error

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return np.where(linear_output >= 0, 1, 0)

# Custom metric calculation for scratch model
def compute_scratch_metrics(y_true, y_pred):
    accuracy = np.mean(y_true == y_pred)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    return accuracy, precision, recall, f1

# Train & evaluate Scratch Model
scratch = PerceptronScratch(learning_rate=0.01, epochs=20)
```

```

scratch.fit(X_train, y_train)
y_pred_scratch = scratch.predict(X_test)
acc_s, prec_s, rec_s, f1_s = compute_scratch_metrics(y_test, y_pred_scratch)

# --- Library Implementation ---
lib = Perceptron(max_iter=20, eta0=0.01, random_state=42, shuffle=False)
lib.fit(X_train, y_train)
y_pred_lib = lib.predict(X_test)

acc_l = accuracy_score(y_test, y_pred_lib)
prec_l = precision_score(y_test, y_pred_lib)
rec_l = recall_score(y_test, y_pred_lib)
f1_l = f1_score(y_test, y_pred_lib)

# --- Terminal Output ---
print("=" * 47)
print("          PERCEPTRON METRICS RESULTS          ")
print("=" * 47)
print(f"{'Metric':<15} | {'From Scratch':<12} | {'Scikit-Learn':<12}")
print("-" * 47)
print(f"{'Accuracy':<15} | {acc_s:<12.4f} | {acc_l:<12.4f}")
print(f"{'Precision':<15} | {prec_s:<12.4f} | {prec_l:<12.4f}")
print(f"{'Recall':<15} | {rec_s:<12.4f} | {rec_l:<12.4f}")
print(f"{'F1-Score':<15} | {f1_s:<12.4f} | {f1_l:<12.4f}")
print("=" * 47)

```

OUTPUT:

=====		
PERCEPTRON METRICS RESULTS		
=====		
Metric	From Scratch	Scikit-Learn

Accuracy	1.0000	1.0000
Precision	1.0000	1.0000
Recall	1.0000	1.0000
F1-Score	1.0000	1.0000
=====		